**基于 BitVLA 的高效长时序 Vision-Language-Action 模型复现与创新设计**

| 汇报时间 | 汇报人 |
| --- | --- |
| 2026.03.15 | 何宁秋 |

# 1. 阶段目标说明

## 1.1近期任务

上一阶段主要完成了两部分工作：

第一，系统梳理了 VLA 的发展脉络，从模块化方法逐步过渡到端到端策略、Foundation VLA 和生成式策略，建立了对整个领域技术路线的整体理解；

第二，在调研多个开源项目后，将当前更适合做论文创新和实验复现的方向聚焦到三类代表方法：

- **3D Diffusion Policy** 代表生成式策略方向，
- **InternVLA** 代表 Foundation VLA 方向，
- **BitVLA** 代表高效轻量化 VLA 方向。

其中 BitVLA 由于模型规模较小、实验成本低、开源完整、易于快速跑通，适合作为当前阶段最优先复现和开展创新设计的 baseline。

## VLA研究主线地图

- RT-2（2023）
  代表意义：把 VLM/VLA 概念真正打响，强调“把网络知识迁移到机器人控制”。论文在 arXiv 公开，但不是你现在最适合复现的对象。
- OpenVLA（2024）
  代表意义：开源 VLA baseline，7B 参数、970k real-world demos，是目前很多开源工作对齐和比较的核心对象。官方论文与官方 GitHub 都公开。
- BitVLA（2025）
  代表意义：把“原生低比特 VLA”推到台前，主打 1-bit / 1.58-bit 部署友好路线。官方论文与官方代码都公开。
- SmolVLA（2025）
  代表意义：小模型、低成本、社区数据、消费级硬件部署，是另一条很强的轻量化路线。官方博客、文档和代码都在 Hugging Face / LeRobot 体系里。

## 论文清单

- 第一组：精读

RT-2 (2023)：VLA 概念标志性论文。

OpenVLA (2024)：开源强 baseline。

BitVLA (2025)：你的当前主线。

SmolVLA (2025)：轻量化另一核心参照。

- 第二组：和baseline文献的创新高度相关

FAST：动作高效 tokenization。

VQ-VLA：向量量化动作 tokenizer。

ActDistill：轻量 student 蒸馏。

VITA-VLA：action expert distillation。

EfficientVLA：training-free 加速。

RetoVLA：轻量化同时保住空间 reasoning。

- 第三组：综述类

Vision-Language-Action Models for Robotics: A Review（2025）

Large VLM-based Vision-Language-Action Models for Robotic Manipulation（2025）

A Survey on Vision-Language-Action Models: An Action Tokenization Perspective（2025）

A Survey on Efficient Vision-Language-Action Models（2025）

Efficient Vision-Language-Action Models for Embodied Manipulation: A Systematic Survey（2025）

# 轻量化/高效 VLA 的主要技术路线

## A. 参数压缩/量化路线

核心代表：BitVLA、EfficientVLA、RetoVLA

BitVLA：做的是原生低比特参数设计，核心卖点是 1-bit VLA 与蒸馏感知训练。

EfficientVLA：强调 training-free 的加速与压缩，更像推理侧优化框架。

RetoVLA：轻量化同时尽量保住空间推理能力，思路是“轻量化不能把空间理解也一起削没”。

这条线适合做的创新方向有：
分层异构量化、低比特下空间推理保持、记忆模块与低比特耦合。

## B. 小模型/低成本训练路线

核心代表：SmolVLA

SmolVLA：450M 量级，目标非常明确，就是让 VLA 能在单 GPU 训练、消费级硬件部署，还用了异步推理栈提升响应。

这条线适合做的创新方向有：
小模型 + 更强泛化、小模型 + 记忆、小模型 + 低成本任务适配。

## C. 蒸馏路线

核心代表：ActDistill、VITA-VLA

ActDistill：把大 VLA 的动作能力蒸馏到轻量模型上，强调 action-guided distillation。

VITA-VLA：用 action expert distillation 教会 VLM 去 act，思路也很适合“轻量 student + 强 teacher”的体系。

这条线适合做的创新方向有：
BitVLA + 动作蒸馏、低比特 VLA + expert teacher、蒸馏降低训练成本。

### D. 动作 token/表示压缩路线

核心代表：FAST、VQ-VLA、FASTer

FAST：高效动作 tokenization，强调让 autoregressive VLA 更适合高频、灵巧操作，并能显著减少训练代价。

VQ-VLA：通过 vector-quantized action tokenizer 改善性能、推理速度和长时任务能力。

FASTer：继续沿“动作 tokenization 的效率与泛化”往前推。

这条线和 BitVLA 很配，因为 BitVLA 主要压的是参数和视觉主干，而动作表示空间仍然有很大文章可做。

### E. 感知稀疏化/token减负路线

核心代表：SemanticVLA

SemanticVLA：强调 semantic-aligned sparsification and enhancement，本质上是在视觉 token/感知表征上做“删冗余、保语义”。

这条线适合做：
低比特 + 稀疏视觉 token、注意力预算优化、任务相关 token 筛选。

## 轻量化VLA小结

### 5个技术路线

| 路线 | 代表论文 |
| --- | --- |
| 模型量化 | BitVLA |
| 小模型 | SmolVLA |
| 蒸馏 | ActDistill / VITA-VLA |
| 动作压缩 | FAST / VQ-VLA |
| token稀疏 | SemanticVLA |

### BitVLA 可创新点地图

| 创新方向 | 改进模块 | 研究问题 | 为什么值得做 | 实验建议 |
| --- | --- | --- | --- | --- |
| **Memory-Enhanced BitVLA** | LLM内部 / policy | 低比特模型如何处理 long-horizon task | BitVLA主要针对效率，没有针对长序列任务优化 | LIBERO long horizon |
| **Action Token Compression** | action tokenizer | 256 action bins 是否冗余 | BitVLA只压缩模型，没有压动作空间 | 动作token数 vs success rate |
| **Latent Action Representation** | action head | 是否可以学习 latent action | 可以减少序列长度，提高推理速度 | action latent vs discrete |

| 创新方向 | 改进模块 | 研究问题 | 为什么值得做 | 实验建议 |
| --- | --- | --- | --- | --- |
| **Distilled BitVLA** | training pipeline | 是否可以通过teacher提升性能 | 蒸馏是当前VLA热门方向 | teacher OpenVLA |
| **Hierarchical BitVLA** | planning module | 高层规划 + 低层控制 | 当前VLA没有层次结构 | subtask success |
| **Sparse Vision Tokens** | vision encoder | 是否可以减少视觉token | 视觉token占大量计算 | token pruning |
| **Spatial Reasoning Preservation** | multimodal alignment | 量化是否损失空间理解 | 低比特容易损失几何信息 | 3D manipulation |
| **Adaptive Quantization** | quantization layer | 不同层是否需要不同bit | BitVLA统一bit | layer-wise quantization |
| **Low-cost Training BitVLA** | training pipeline | 是否能减少训练成本 | BitVLA训练流程复杂 | fewer training steps |

考虑的创新点：

- 创新 1：压缩和优化动作表示
- 创新 2：增强时序感知
- 创新 3：改善动作生成质量

BitVLA 已证明低比特 VLA 可行，但其公开实现仍存在三个瓶颈：
动作离散化过于粗糙、长时任务缺少显式时序建模、chunked control 的动作输出仍可进一步优化。
因此提出一套面向低比特 VLA 的动作表示与时序控制增强框架.....

## 1.2本阶段目标

本阶段的目标：

**以 BitVLA 为 baseline，完成论文理解、代码跑通、创新入口分析，并围绕高效动作表示与长时序建模设计后续实验路线。**

- 从“VLA 脉络理解”转向“具体 baseline 落地”
- 从“看论文”转向“找可实现的创新点”
- 从“方法分类”转向“形成自己的论文故事”

研究整体路线图：

| 阶段 | 目标 | 任务 | 输出 |
| --- | --- | --- | --- |
| **阶段1** | 完全理解BitVLA | 阅读论文+跑通demo | 模型结构图 |
| **阶段2** | 找创新入口 | 分析代码结构 | 创新设计文档 |
| **阶段3** | 创新1实现 | Adaptive Action Tokenization | 第一个实验结果 |

| 阶段 | 目标 | 任务 | 输出 |
| --- | --- | --- | --- |
| **阶段4** | 创新2实现 | Temporal History Fusion | long-horizon实验 |
| **阶段5** | 创新3实现 | Hybrid Action Head | 控制平滑实验 |
| **阶段6** | 实验总结 | ablation + benchmark | 完整论文 |

论文三点创新结构：

| 创新点 | 研究问题 | 方法 | 代码改动 | 实验 |
| --- | --- | --- | --- | --- |
| **创新1** Adaptive Action Tokenization | 256 action bins 是否冗余 | 自适应分桶 | action tokenizer | token数量vs成功率 |
| **创新2** Temporal History Fusion | 低比特模型如何处理 long-horizon | 历史帧融合 | dataset + encoder | LIBERO-Long |
| **创新3** Hybrid Residual Action Head | chunk action 是否不够平滑 | 离散+连续残差 | action head | smoothness |

论文故事：

**低比特VLA效率很好，但动作表示、时序建模和控制输出仍有优化空间。**

指南：

| 创新点 | 修改文件 | 修改内容 | 难度 |
| --- | --- | --- | --- |
| Adaptive Tokenization | `bitvla/bitnet_action_tokenizer.py` | 自适应bin | ⭐ |
| | `bitvla_transform.py` | action encode/decode | ⭐ |
| Temporal Fusion | `dataset loader` | 加历史帧 | ⭐⭐ |
| | `bitvla_for_action_prediction.py` | temporal module | ⭐⭐⭐ |
| Hybrid Action Head | `action_heads.py` | residual head | ⭐⭐⭐ |
| | `openvla_utils.py` | action head调用 | ⭐⭐ |

# 2. BitVLA论文与方法概述

## 2.1 基础信息

**选择原因：**

选择 BitVLA 作为当前 baseline，基于四点考虑：

第一，它是轻量级 VLA，模型更小，复现门槛相对较低。
第二，它的论文核心问题很明确：**在保证操控性能的同时，显著降低 VLA 的显存占用和推理延迟。**
第三，它不是单纯后量化，而是在训练过程中就引入低比特约束，这使它本身就天然具备“高效 VLA”研究价值。
第四，它非常适合作为创新母体，因为低比特 VLA 的效率已经不错，但在**动作表示、时序建模和动作输出稳定性**方面仍然存在进一步改进空间。

**BitVLA 的核心思想**：

BitVLA 是论文中提出的**首个 fully native 1-bit Vision-Language-Action 模型**，其核心思想是：
不是先训练一个全精度大模型再事后压缩，而是在训练阶段就把“高效部署”纳入模型设计目标，通过低比特化和训练协同设计来实现效率与性能平衡。论文中所有主干参数都被限制为 ternary，即 `{-1, 0, 1}`，语言主干来自 1-bit LLM BitNet b1.58 2B4T；同时作者又提出 **Quantize-then-Distill**，把视觉编码器进一步压到 **1.58-bit 权重 + INT8 激活**，从而减少视觉分支开销。

论文显示，BitVLA 在仿真和真实机器人任务上能达到接近 OpenVLA-OFT 的性能，同时把模型显存降低到 **1.4GB**，相对 OpenVLA-OFT 实现 **11× memory reduction**，端到端推理速度提升约 **4.4×**。

**BitVLA 的训练流程：**

BitVLA 的训练流程可以分成三阶段：

- **第一阶段：Multimodal Training**
  先把 1-bit LLM 和全精度视觉编码器连接起来，完成视觉-语言对齐。
- **第二阶段：Quantize-then-Distill**
  把视觉编码器量化到 1.58-bit，并用全精度 teacher 的中间表征来蒸馏 student，以缓解量化带来的表征偏移。
- **第三阶段：Robotics Training**
  在机器人轨迹数据上做动作预测训练，形成真正的 VLA policy。论文中使用基于 Open X-Embodiment 整理的约 **1M 机器人样本**进行预训练。每个动作维度被离散化为 **256 个 bins**，并结合 action chunking 进行并行解码。

以上阶段暗示的几个问题：

- 256 个 action bins 是否存在冗余？
- 低比特表示在 long-horizon 任务里是否会丢失历史信息？
- chunk action 输出是否会影响动作平滑性？

对应前期规划的三个创新点。

## 2.2 BitVLA 的实验结果与启发

**性能与效率**：

论文结果表明，BitVLA 在 LIBERO 上表现很强。
在 small-size 模型里，BitVLA 只有 **3.0B** 参数、**1.4GB** 内存，但平均成功率达到 **96.0**，其中 **LIBERO-Long** 达到 **92.8**；未做机器人预训练的版本在 LIBERO-Long 上只有 **87.6**，说明大规模机器人预训练对于长时序任务非常关键。
与 π0 相比，BitVLA 在 LIBERO-Long 上高出 **7.6%**，说明它在长时序操作上已经展现出明显优势。

在推理效率上，BitVLA 报告的结果是 **73 ms latency**、**341.1 Hz throughput**，快于 OpenVLA-OFT+、π0 以及 Diffusion Policy。这个结果说明：
**BitVLA 的主要优势不是单纯“模型小”，而是“以低比特训练为核心的原生高效设计”，效率收益是真正能落到端到端控制上的。**

对后续创新的启发

BitVLA 已经证明了一件事：
高效 VLA 是可行的。

但论文也留下了很明显的可继续做的空间：

第一，当前动作离散化采用固定 **256 bins**，这可能不是最优的动作表示。
第二，虽然 BitVLA 在 LIBERO-Long 上表现不错，但 long-horizon 建模仍然是 VLA 的核心难题之一。
第三，BitVLA 采用 chunked action parallel decoding，这对吞吐有利，但可能会带来控制平滑性和残差校正不足的问题。

因此，BitVLA 非常适合作为一个“效率已经做得不错，但还没有把动作表示、时序记忆、控制输出做到最优”的 baseline。

## 2.3 论文创新主线

**核心论文故事：**

低比特 VLA 已经在效率上取得明显优势，但其动作表示、历史建模与控制输出仍有优化空间。

围绕 BitVLA 做三点改进，形成一条统一主线：

Efficient Long-Horizon Vision-Language-Action Models

三点创新都可以解释为：**为了让低比特 VLA 在保持效率的前提下，更好地完成长时序操控。**

---

## 2.4 创新点设计

### 创新点1：Adaptive Action Tokenization

**问题**

BitVLA 当前将每个动作维度固定离散为 256 个 bins。但不同动作维度、不同任务阶段对离散精度的需求并不相同，固定均匀分桶可能存在冗余。

**想法**

将固定 256-bin 动作离散化，改为**自适应动作分桶**。
可以考虑：

- 依据动作分布密度做非均匀分桶
- 对不同 action dimension 使用不同 bin 数
- 在 coarse-to-fine 的框架下进行分层 tokenization

**预期收益**

- 缩短 action token length

- 降低无效离散精度带来的计算开销
- 在保持成功率的同时进一步提升推理效率

**对应代码位置**

- `bitvla/bitnet_action_tokenizer.py`
- `bitvla_transform.py`

**对应实验**

- 256 vs 128 vs 64
- 比较成功率、token 长度、推理延迟之间的关系

## 创新点2：Temporal ssHistory Fusion

**问题**

低比特模型虽然效率高，但在 long-horizon manipulation 中，历史信息保持能力可能不足。而长时序任务本身就是 VLA 的核心难点之一。

**想法**

在 BitVLA 中引入**历史帧融合模块**，将短窗口历史观测显式送入模型，而不只依赖当前单步观测。
可以尝试：

- 直接拼接 history frames
- 增加 temporal fusion block
- 对历史视觉 token 做轻量聚合，再与当前状态共同输入

**预期收益**

- 提升 long-horizon 任务成功率
- 缓解低比特表示可能带来的时序信息丢失
- 强化任务分解和阶段记忆能力

**对应代码位置**

- S`dataset loader`
- `bitvla_for_action_prediction.py`

**对应实验**

- history = 0 / 1 / 2
- 重点看 LIBERO-Long 上的成功率变化

## 创新点3：Hybrid Residual Action Head

**问题**

BitVLA 使用 action chunking 提高吞吐，但 chunk 输出可能带来动作不够平滑、控制局部误差无法及时修正的问题。

**想法**

设计**离散动作 + 连续残差修正**的 hybrid action head。
也就是保留原本离散 token 的高效建模优势，同时增加一个小型 residual head 来补偿细粒度连续控制误差。

**预期收益**

- 提升动作平滑性
- 增强控制稳定性
- 降低离散动作带来的量化误差

**对应代码位置**

- `action_heads.py`
- `openvla_utils.py`

**对应实验**

- discrete vs hybrid
- 对比 success rate 与 action smoothness

## 2.5 方法结构图

设计逻辑图：

```
Input Images
   ↓
Vision Encoder
   ↓
Connector
   ↓
BitNet LLM
   ↓
Temporal Fusion
   ↓
Adaptive Action Tokenizer
   ↓
Hybrid Action Head
   ↓
Robot Control
```

这张图的核心含义是：

- **BitVLA** 作为统一 backbone
- **Temporal Fusion** 解决长时序信息利用
- **Adaptive Action Tokenizer** 解决动作表示冗余
- **Hybrid Action Head** 解决控制输出平滑性

这样三点创新从输入到输出形成一条连续的改进链路。

# 3. 实验设计与指标

## 3.1 实验类型

实验部分可以分成五类：

1. **Baseline 实验**：先复现原始 BitVLA
2. **Token 实验**：比较不同动作分桶数
3. **Memory 实验**：比较是否加入历史信息
4. **Head 实验**：比较原始动作头和混合动作头
5. **Ablation 实验**：分析每个模块单独贡献

框架图：

| 实验类型 | 目的 | 对比 |
| --- | --- | --- |
| Baseline | 基准 | BitVLA |
| Token实验 | 动作压缩 | 256 vs 128 vs 64 |
| Memory实验 | 时序能力 | history=0,1,2 |
| Head实验 | 控制平滑 | discrete vs hybrid |
| Ablation | 模块贡献 | 去掉模块 |

## 3.2 核心评测指标

4 个关键指标：

- **Success Rate**：任务成功率
- **Inference Latency**：推理速度
- **Action Smoothness**：动作平滑性
- **Token Length**：action token 数量

其中：

- 创新点1 重点看 Token Length + Latency + Success Rate
- 创新点2 重点看 LIBERO-Long Success Rate
- 创新点3 重点看 Smoothness + Success Rate

这样每个创新点都有明确指标对应，不会显得实验设计松散。

---

## 3.3 运行环境

当前租赁的服务器配置是：

- GPU：**RTX 4090D 24GB（1卡）**

- CPU：**16 vCPU Xeon 8481C**
- 内存：**80GB**
- 系统：**Ubuntu 22.04**
- Python：**3.10**
- PyTorch：**2.1.0**
- CUDA：**12.1**

AutoDL服务器：[https://www.autodl.com/home](https://www.autodl.com/home)

# 4. 下一阶段执行计划

## 4.1 执行步骤

**第一步：梳理整体代码框架及跑通应用评测入口**

目标：

- 梳理全流程代码，创新点考虑；
- 实验服务器及环境准备；
- 跑通应用demo

输出：

- 跑通流程

20260315备注：当前处于这一步骤

**第二步：跑通原始 BitVLA baseline**

目标：

- 完成环境依赖检查
- 跑通 demo 或最小训练/推理流程
- 梳理模型结构图和数据流

输出：

- baseline 运行记录
- 模型流程图
- 可修改模块清单

**第三步：做 Adaptive Action Tokenization**

目标：

- 先在最小改动下验证 action token 压缩是否有效
- 建立第一个“有结果的创新实验”

输出：

- 256 / 128 / 64 bins 对比结果
- token length 与 latency 曲线
- 初版实验结论

**第四步：做 Temporal History Fusion**

目标：

- 在 LIBERO-Long 或类似 long-horizon 任务上验证历史信息融合
- 观察低比特模型的时序建模瓶颈

输出：

- history=0/1/2 的 long-horizon 对比
- 失败案例分析

**第五步：做 Hybrid Residual Action Head**

目标：

- 观察动作输出平滑性是否改善
- 看是否能进一步提高控制稳定性

输出：

- smoothness 对比
- discrete vs hybrid 结果表

**第六步：做 ablation 与完整论文组织**

目标：

- 把三个点连成一条主线
- 补齐消融与可视化分析
- 形成完整论文框架